

1. Files used

Minimum flashing folder:

```
flash_j18.bat
SAMD21J18_Blinky.ino.bin
```

Optional config-file flow:

```
flash_j18.cfg
flash_j18_runner.bat
SAMD21J18_Blinky.ino.bin
```

Keep only one .bin in the flashing folder unless you intentionally want the first alphabetic .bin to be flashed.

2. SWD wiring

ST-LINK side		SAMD21 target side	
ST-LINK	SWDIO	SAMD21	PA31 / SWDIO
ST-LINK	SWCLK	SAMD21	PA30 / SWCLK
ST-LINK	GND	SAMD21	GND
ST-LINK	VTref	Target	3.3 V sense
ST-LINK	NRST	SAMD21	RESET, optional

Power the target board normally. The ST-LINK VTref pin should sense the target 3.3 V rail. Do not wire RESET to 5 V. Pull RESET to 3.3 V through about 10 kOhm if it does not idle high.

3. Install tools and get Arduino SAMD files

Run these from PowerShell. Restart PowerShell after installing toolchain packages so PATH updates are picked up.

```
winget install -e --id Git.Git
winget install -e --id ArduinoSA.CLI
winget install -e --id xpack-dev-tools.openocd-xpack
```

Install the ST-LINK Windows driver if Windows/OpenOCD cannot open the ST-LINK:

```
Install STSW-LINK009
Unplug ST-LINK
Plug ST-LINK back in
```

Check the installed tools:

```
git --version
arduino-cli version
openocd --version
```

Clone the Arduino SAMD repo for bootloaders, linker scripts, board definitions, and source reference:

```
cd C:\dev
git clone https://github.com/arduino/ArduinoCore-samd.git
```

Install the Arduino SAMD board package through Arduino CLI, or install Arduino SAMD Boards through Boards Manager in the Arduino IDE.

CLI calls:

```
arduino-cli core update-index
arduino-cli core install arduino:samd
arduino-cli board listall samd
```

4. Test sketch: PA15 blinky

Create this folder and sketch file:

```
SAMD21J18_Blinky/
  SAMD21J18_Blinky.ino
```

Use board target Arduino Zero (Native USB) from Arduino SAMD Boards. This sketch directly writes SAMD21 port registers, so it avoids relying on the Arduino Zero pin numbering table. To output binaries in the IDE do *Sketch -> Export Compiled Binary*

```
#include <Arduino.h>

#define PORT_A 0
#define PORT_B 1

#define PA00 PORT_A, 0
#define PA01 PORT_A, 1
#define PA02 PORT_A, 2
#define PA03 PORT_A, 3
#define PA04 PORT_A, 4
#define PA05 PORT_A, 5
#define PA06 PORT_A, 6
#define PA07 PORT_A, 7
#define PA08 PORT_A, 8
#define PA09 PORT_A, 9
#define PA10 PORT_A, 10
#define PA11 PORT_A, 11
#define PA12 PORT_A, 12
#define PA13 PORT_A, 13
#define PA14 PORT_A, 14
#define PA15 PORT_A, 15
#define PA16 PORT_A, 16
#define PA17 PORT_A, 17
#define PA18 PORT_A, 18
#define PA19 PORT_A, 19
#define PA20 PORT_A, 20
#define PA21 PORT_A, 21
#define PA22 PORT_A, 22
#define PA23 PORT_A, 23
#define PA24 PORT_A, 24
#define PA25 PORT_A, 25
#define PA27 PORT_A, 27
#define PA28 PORT_A, 28
#define PA30 PORT_A, 30
#define PA31 PORT_A, 31

#define PB00 PORT_B, 0
#define PB01 PORT_B, 1
#define PB02 PORT_B, 2
#define PB03 PORT_B, 3
```

```

#define PB04 PORT_B, 4
#define PB05 PORT_B, 5
#define PB06 PORT_B, 6
#define PB07 PORT_B, 7
#define PB08 PORT_B, 8
#define PB09 PORT_B, 9
#define PB10 PORT_B, 10
#define PB11 PORT_B, 11
#define PB12 PORT_B, 12
#define PB13 PORT_B, 13
#define PB14 PORT_B, 14
#define PB15 PORT_B, 15
#define PB16 PORT_B, 16
#define PB17 PORT_B, 17
#define PB22 PORT_B, 22
#define PB23 PORT_B, 23
#define PB30 PORT_B, 30
#define PB31 PORT_B, 31

#define BLINK_PORT_PIN PA15

static inline void pinOutput(uint8_t port, uint8_t pin) {
    PORT->Group[port].DIRSET.reg = 1ul << pin;
}

static inline void pinLow(uint8_t port, uint8_t pin) {
    PORT->Group[port].OUTCLR.reg = 1ul << pin;
}

static inline void pinToggle(uint8_t port, uint8_t pin) {
    PORT->Group[port].OUTTGL.reg = 1ul << pin;
}

void setup() {
    pinOutput(BLINK_PORT_PIN);
    pinLow(BLINK_PORT_PIN);
}

void loop() {
    pinToggle(BLINK_PORT_PIN);
    delay(500);
}

```

5. Compile the sketch

IDE route: select Arduino Zero (Native USB), compile/export the binary, then copy the exported .bin into your flashing folder.

CLI route, normal Arduino build:

```

arduino-cli compile `
--fqbn arduino:samd:arduino_zero_native `
--build-path .\build `
--export-binaries `
.\SAMD21J18_Blinky

```

CLI route, recommended no-bootloader SWD build:

```

arduino-cli compile `
--fqbn arduino:samd:arduino_zero_native `
--build-path .\build-noboot `
--export-binaries `
--build-property build.ldscript=linker_scripts/gcc/flash_without_bootloader.ld `

```

```
.\SAMD21J18_Blinky
```

Copy the produced .bin into the flashing folder beside flash_j18.bat OR flash_j18.cfg.

6. Recommended Windows flashing method: direct batch file

Save this as flash_j18.bat in the same folder as the compiled .bin.

Then double-click flash_j18.bat. This batch will try to flash the bootloader version of the arduino sketches based on their “,with_bootloader.bin” convention first or otherwise it assumes you are flashing just the sketch alone, already with the bootloader.

```
@echo off
setlocal EnableExtensions
cd /d "%~dp0"

set "BIN_FILE="
set "FLASH_ADDR="

for /f "delims=" %%F in ('dir /b /a-d *.with_bootloader.bin 2^>nul ^| sort') do (
    set "BIN_FILE=%%F"
    set "FLASH_ADDR=0x00000000"
    goto :found_bin
)

for /f "delims=" %%F in ('dir /b /a-d *.bin 2^>nul ^| sort') do (
    set "BIN_FILE=%%F"
    set "FLASH_ADDR=0x00002000"
    goto :found_bin
)

echo ERROR: no .bin file found in this folder.
echo Put this .bat next to the compiled firmware .bin.
echo.
pause
exit /b 1

:found_bin
echo Flashing firmware: "%BIN_FILE%"
echo Flash address: %FLASH_ADDR%
echo.

openocd ^
    -f interface/stlink.cfg ^
    -c "transport select swd" ^
    -c "adapter speed 400" ^
    -c "set CHIPNAME at91samd21j18" ^
    -f target/at91samdXX.cfg ^
    -c "reset_config none" ^
    -c "cortex_m reset_config sysresetreq" ^
    -c "init" ^
    -c "targets" ^
    -c "reset halt" ^
    -c "at91samd bootloader 0" ^
    -c "program {%BIN_FILE%} verify reset exit %FLASH_ADDR%"

set "OPENOCD_EXIT=%ERRORLEVEL%"

echo.
```

```
echo OpenOCD exit code: %OPENOCD_EXIT%
echo.
pause
exit /b %OPENOCD_EXIT%
```

7. Alternate method: OpenOCD config file

Save this as `flash_j18.cfg` beside the compiled `.bin`. Run it from that folder with this command:

```
openocd -f .\flash_j18.cfg

set BIN_FILES [lsort [glob -nocomplain *.bin]]

if {[llength $BIN_FILES] == 0} {
    echo "ERROR: no .bin file found in this folder"
    shutdown
}

set BIN_FILE [lindex $BIN_FILES 0]
echo "Flashing first .bin found: $BIN_FILE"

source [find interface/stlink.cfg]
transport select swd

adapter speed 400

set CHIPNAME at91samd21j18
source [find target/at91samdXX.cfg]

reset_config none
cortex_m reset_config sysresetreq

init
targets

reset halt

at91samd bootloader 0

program $BIN_FILE verify reset exit 0x00000000
```

If you already have a bootloader, use this config so the addressing is correct.

```
# flash_j18_existing_bootloader.cfg
# OpenOCD config for flashing an Arduino SAMD app .bin when the bootloader already exists.
#
# Save this beside the compiled .bin.
#
# Run from that folder:
#   openocd -f .\flash_j18_existing_bootloader.cfg
#
# Memory layout:
#   Bootloader: 0x00000000 -> 0x00001FFF
#   App/sketch: 0x00002000 onward

set BIN_FILES [lsort [glob -nocomplain *.bin]]

if {[llength $BIN_FILES] == 0} {
    echo "ERROR: no .bin file found in this folder"
    shutdown
}

set BIN_FILE [lindex $BIN_FILES 0]
```

```

echo "Flashing first .bin found: $BIN_FILE"
echo "Flash address: 0x00002000"

source [find interface/stlink.cfg]
transport select swd

adapter speed 400

set CHIPNAME at91samd21j18
source [find target/at91samdXX.cfg]

reset_config none
cortex_m reset_config sysresetreq

init
targets

reset halt

# Keep the existing bootloader region protected if it is already protected.
# Do not run "at91samd bootloader 0" here unless you intentionally want to unlock it.

program $BIN_FILE verify reset exit 0x00002000

```

Optional launcher for the config-file method. Save as flash_j18_runner.bat:

```

@echo off
cd /d "%~dp0"
openocd -f flash_j18.cfg
pause

```

8. Flash address choice

- 0x00000000: use this for a no-bootloader application image. This is the recommended SWD test path. I.e. flash the `x.with_bootloader.bin` binary option that Arduino produces.
- 0x00002000: use this only if the binary was linked for the Arduino SAMD bootloader offset i.e. if you don't want to replace the existing bootloader.

To use the bootloader offset, change only the final program address:

```
0x00000000 -> 0x00002000
```

9. Expected successful output

```

Flashing first .bin found: "SAMD21J18_Blinky.ino.bin"
Open On-Chip Debugger 0.12.0
adapter speed: 400 kHz
Info : STLINK V2J29S7
Info : Target voltage: 3.2 V
Info : SWD DPIDR 0x0bc11477
Info : [at91samd21j18.cpu] Cortex-M0+ r0p1 processor detected
Info : [at91samd21j18.cpu] Examination succeed
** Programming Started **
Info : SAMD MCU: SAMD21J18A (256KB Flash, 32KB RAM)
** Programming Finished **
** Verify Started **
** Verified OK **
OpenOCD exit code: 0

```

A reset-time HardFault after `verified OK` can still happen if the firmware was linked for the wrong start address, if the clock/startup assumptions do not match the board, or if reset lands somewhere invalid. The flash step itself succeeded if the verification passed.

10. Common errors

Error	Fix
Debug adapter does not support <code>hla_swd</code> transport	Use transport select <code>swd</code> , not <code>hla_swd</code> .
Error: open failed	Install STSW-LINK009, unplug/replug ST-LINK, close STM32CubeProgrammer, or use Zadig/WinUSB if OpenOCD still cannot access it.
No SWD DPIDR / target not examined	Check power, VTref, GND, SWDIO, SWCLK, and reduce adapter speed to 100 kHz.
Verified OK but no blink	Check PA15 wiring and confirm the binary was linked for the address you flashed. We don't have an indicator LED on the board yet so you need to test this manually

Slower SWD test:

```
-c "adapter speed 100"
```